Hitesh Rathore(HR) Notes
LINKEDIN LINK:
https://www.linkedin.com/in/hitesh-rathore-[illegible]/

1) contenteditable — MAKE ELEMENTS EDITABLE

The `contenteditable` attribute makes specific elements editable by the user.

WHAT IS contenteditable?

Adds inline editing ability to a specific element.

Browser enables:
- ✓ text selection
- ✓ caret system
- ✓ editing commands
- ✓ paste support
- ✓ formatting (bold, italic, etc.)

SIMPLE EXAMPLE (NO CSS)

```
<div contenteditable="true">
  Edit this text...
</div>
```

LIVE PREVIEW

Edit this text...
(Click and start editing)

WHAT HAPPENS?

User can:
- ✓ edit text
- ✓ delete content
- ✓ type anywhere
- ✓ paste content
- ✓ format text

Directly inside that element

IMPORTANT VALUES

Value	Meaning
true	Element is editable
false	Element is NOT editable
plaintext-only	Only plain text allowed (no formatting)

INTERNAL FLOW

→ User focuses editable element
↓
Browser activates editor for that element
↓
Selection engine activated
↓
Caret rendering activated
↓
Editing commands enabled
↓
Content changes reflect in the DOM instantly

REAL WORLD USE CASES

Inline Comment Editing | Editable Table Cells | Note Taking Apps | Chat Input Areas | Custom Rich Text Editors

IMPORTANT NOTES

- Works only on the element where it is applied.
- Changes are reflected in the LIVE DOM.
- Developer must handle saving the data.
- Use plaintext-only if you don't want rich formatting.
- Fully supported in all modern browsers.

PERMANENT SAVE (IMPORTANT)

By default, changes made using contenteditable are temporary (in memory). To make them permanent, you must save the content using JavaScript.

1. Get content (innerHTML / innerText) → 2. Save to Database or Local Storage → 3. Reload / Visit Again Content is restored

STORAGE OPTIONS
- Database (Server-side)
- Local Storage
- IndexedDB, etc.

⚠ Without saving, content will be lost after page refresh.

2) designMode — ENTIRE PAGE BECOMES EDITABLE

This is one of the CRAZIEST hidden browser features.

WHAT IS designMode?

`document.designMode = 'on';`

When this runs:
Entire webpage becomes editable
Browser activates for the WHOLE document:
- ☑ live editing engine
- ☑ text selection engine
- ☑ caret system
- ☑ editing commands
- ☑ formatting engine

SIMPLE EXAMPLE (NO CSS)

```
<h1>Hello World</h1>

<p>Try editing this webpage.</p>

<script>
 document.designMode = 'on';
</script>
```

WHAT HAPPENS?

Now user can:
- ☑ edit text
- ☑ delete content
- ☑ type anywhere
- ☑ paste content
- ☑ format entire page

Directly on the webpage

LIVE PREVIEW (CONCEPT)

Hello World
Try editing this webpage.
User can edit anything on page!

IMPORTANT

This changes:
</> LIVE DOM ONLY
Refresh:
everything lost
unless saved manually.

INTERNAL FLOW

designMode ON
↓
Browser activates document editor mode
↓
Entire DOM becomes editable
↓
Selection engine activated
↓
Caret rendering activated
↓
Editing commands enabled

REAL WORLD USE CASES

- **Gmail Old Editor** Earlier Gmail used designMode for composing emails.
- **Old Blog Editors** Blogger (old version) and other platforms used designMode for post editing.
- **CMS Systems** Many old Content Management Systems used designMode for page editing.
- **Internal Admin Panels** Some internal tools used it for quick content editing on the page.
- **WYSIWYG Editors (Old)** Early rich text editors relied on designMode internally.
- **Prototyping & Experiments** Developers used designMode for quick inline editing prototypes.

IMPORTANT NOTES

- Affects the WHOLE document, not just one element.
- All elements become editable.
- Not recommended for modern production editors.
- Use contenteditable for specific regions, use designMode only for full-page editing.
- Supported in all modern browsers.
- Changes are in memory (LIVE DOM).
- Developer must handle saving the data.

PERMANENT SAVE (IMPORTANT)

By default, changes made using designMode are temporary (in memory). To make them permanent, you must save the content using JavaScript.

1. Get content (innerHTML / innerText) → 2. Save to Database or Local Storage → 3. Reload / Visit Again Content is restored →

STORAGE OPTIONS
- Database (Server-side)
- Local Storage
- IndexedDB, etc.

⚠ Without saving, content will be lost after page refresh.

IMPORTANT NOTE

designMode is mostly OLD / LEGACY technology now. Modern production apps usually DO NOT use designMode anymore.

Modern editors use:
- ☑ contenteditable
- ☑ Selection API
- ☑ Range API
- ☑ virtual document models
- ☑ custom editor engines

WHY designMode IS LESS POPULAR NOW? ✗ Edits entire document ✗ Difficult control ✗ Messy DOM ✗ Inconsistent formatting ✗ Browser differences ✗ Security issues ✗ Hard state management

Modern editors build CUSTOM EDITING ENGINES on top of contenteditable with Selection API, Range API and Virtual Document Models.

3) DRAGGABLE ATTRIBUTE — Make Elements Draggable by the User in the Browser

The draggable attribute specifies whether an element can be dragged by the user.

`<img src="photo.png" draggable="true">`

BROWSER SUPPORT

✓ ✓ ✓ ✓ ✓
Supported in all modern browsers.

WHAT IT DOES

- ✓ Enables an element to be dragged.
- ✓ Browser shows a "drag image" (ghost image).
- ✓ Does NOT move or reorder automatically.
- ✓ You must write JavaScript logic to handle drag and drop behavior.

DRAG PREVIEW

DRAGGABLE VALUES

Value	Description
true	Element is draggable
false	Element is NOT draggable
auto (default)	Draggable for some elements (like `<img>`, `<a>`)
plaintext-only	Not valid for draggable (only for contenteditable)

BASIC DRAG FLOW (INTERNAL)

1. Mouse down → 2. dragstart event fires → 3. Browser creates drag image (ghost image) → 4. dragover event on target → 5. drop event on target

IMPORTANT NOTES

- draggable="true" only enables dragging.
- It does not change position, reorder, or swap by itself.
- Text selection usually does not work while dragging (browser gives priority to drag).
- To have both, create custom drag logic using JavaScript.
- Works on most elements, including images and text.
- Native Drag API has limitations on mobile devices.

DRAG EVENTS (JavaScript)

Event	When It Fires	Description
dragstart	When dragging starts	Fires on the dragged element
drag	While dragging	Fires repeatedly while dragging
dragenter	When dragged element enters a valid target	Fires on the target
dragover	While over the target	Fires continuously on the target (must preventDefault() to allow drop)
dragleave	When dragged element leaves the target	Fires on the target
drop	When dropped on target	Fires on the target (handle data drop)
dragend	When dragging ends	Fires on the dragged element

USE CASES (REAL WORLD)

Drag & Drop File Upload | Reorder Items | Kanban Boards (Task Management) | Image Drag & Drop | Text / Element Repositioning | Move Files in Folders | E-commerce (Product Sorting) | UI Builder (Drag Widgets) | Game Elements (Drag Pieces) | Custom Design Tools

IS DRAGGABLE USED IN MODERN WEB?

YES, IT IS STILL USED ✓
- Widely used for basic drag & drop functionality.
- Supported in all modern browsers.
- Used in many tools, dashboards, and web apps.
- Good for simple drag & drop needs.

BUT IT HAS LIMITATIONS ✗
- Not fully reliable on mobile/touch devices.
- Limited control over drag image.
- Default browser behavior is hard to customize.
- Complex apps need more advanced libraries/APIs.

WHAT IS USED IN MODERN APPS INSTEAD / ALONG WITH DRAGGABLE?

For advanced drag & drop, developers use libraries and modern APIs.

1. HTML5 Drag & Drop API (Native - draggable)
- Good for simple use cases
- Works well on desktop
- Easy to implement

2. SortableJS (JavaScript Library)
- Best for sortable lists
- Reordering, swapping
- Highly customizable

3. Interact.js (JavaScript Library)
- Drag, drop, resize, rotate, gesture support
- Very powerful

4. DnD Kit (Modern React Library)
- Advanced drag & drop for React apps
- Used in production

5. Pointer Events API (Modern API)
- More control for custom drag systems
- Better mobile support

draggable="true" only makes an element draggable. You must handle the drag events using JavaScript. It is still used in modern web, but for advanced needs, libraries and modern APIs are preferred.

KEY TAKEAWAY

Use native draggable for simple tasks. Use libraries (SortableJS, DnD Kit, Interact.js) for professional, complex drag & drop features.

4) spellcheck

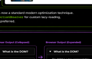
```
<p contenteditable="true" spellcheck="true">
  Type here...
</p>
```

What happens?
- Browser detects spelling mistakes
- Shows red underline
- Provides correction suggestions

WITHOUT JavaScript.

Real World Usage
- ✓ Gmail
- ✓ Notion
- ✓ Google Docs
- ✓ Medium

Browser Output


This is a smple text with wrng spelling.
wrong
wring
wrong
Add to dictionary
Ask Google for suggestions
Search Google for "wrong"

Is it used in modern time?
YES — spellcheck is still widely used in modern websites and web apps. It is built-in in browsers and requires no extra libraries.

If not, then what is used?
For advanced needs, apps also use AI-based grammar checkers like Grammarly, LanguageTool, or custom NLP models for better accuracy and suggestions.

Modern Usage Note: YES — still heavily used in modern websites. Advanced apps also use custom AI-based grammar and spell-check systems for better accuracy.

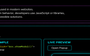
5) loading="lazy"

`<img src="photo.jpg" loading="lazy">`

What happens?
- Browser does NOT load the image immediately.
- It loads ONLY when the image gets close to the viewport.

Why This Is Awesome
- ✓ Performance optimization
- ✓ Bandwidth saving
- ✓ Faster websites
- ✓ Reduced initial loading time

Browser Output (Scroll to see effect)



Modern Usage Note: YES — this is now a standard modern optimization technique. Some advanced websites use `IntersectionObserver` for custom lazy-loading, but native `loading="lazy"` is widely preferred.

6) details + summary

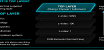
```
<details>
  <summary>Open</summary>
  Hidden content
</details>
```

What happens?
- Browser automatically provides:
- Accordion behavior
- Expand/Collapse logic
- Keyboard accessibility
- Toggle handling

WITHOUT JavaScript.

Real World Usage
- ✓ FAQs
- ✓ Documentation
- ✓ Settings panels
- ✓ Expandable menus
- ✓ Learning platforms

Browser Output (Collapsed) → Browser Output (Expanded)


▶ What is the DOM?
▶ What is CSSOM?
▼ What is the DOM?
The DOM is a tree representation of the webpage created by the browser.
▶ What is CSSOM?

Modern Usage Note: YES — still widely used in modern websites. For complex animations or custom accordion behavior, developers use JavaScript or libraries, but `<details>` is perfect for simple and accessible solutions.

7) <dialog>

`<dialog>` is a native HTML modal / popup system.

WHAT IT DOES

Browser handles everything natively:
- Popup window
- Focus handling
- Background blocking
- ESC close behavior

WITHOUT building everything manually.

BASIC EXAMPLE

```
<button onclick="box.showModal()">
  Open Popup
</button>

<dialog id="box">
  <h1>Hello 👋 </h1>
  <p>This is a dialog box.</p>

  <button onclick="box.close()">
    Close
  </button>
</dialog>
```

LIVE PREVIEW

Open Popup
Hello 👋
This is a dialog box.
Close

INTERNAL FLOW

Button Click
↓
showModal()
↓
Browser moves dialog to TOP LAYER
↓
Background becomes inactive
↓
Focus trapped inside dialog

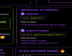
WHAT IS TOP LAYER?

Browser has a special rendering layer:

TOP LAYER

Used for:
- ✓ Dialogs
- ✓ Popovers
- ✓ Fullscreen
- ✓ Dropdown systems

This layer sits above everything, even above the highest z-index.


TOP LAYER (Dialog / Popover / Fullscreen)
z-index: 9999
z-index: 100
z-index: 1
DOM Elements (Normal Flow)
Highest Priority
Lowest Priority

REAL WORLD USE CASES

Login Popup | Confirmation Modal | Payment Popup | Alerts | Settings Window | And many more...

Modern Usage Note: YES — `<dialog>` is a modern, native solution that makes building modals, popups, and overlays easier, accessible, and highly reliable.

8) <popover>

NEW MODERN HTML FEATURE 🔥

`<div popover>` gives you native popup, tooltip, and floating UI without relying on huge JavaScript libraries.

WHAT IS POPOVER?

Browser provides a built-in popup engine to create modern floating interfaces with minimal code.

BASIC EXAMPLE

```
<button popovertarget="menu">
  Open Menu
</button>

<div popover id="menu">
  Hello 👋
</div>
```

LIVE PREVIEW

Open Menu
Hello 👋

BEFORE popover

Developers had to manually create:
- ✗ Absolute positioning
- ✗ z-index battles
- ✗ Click outside detection
- ✗ Open/close state
- ✗ Accessibility
- ✗ Focus management

😫 Huge pain.

NOW?

Browser internally already contains:

POPUP ENGINE inside itself.

You just use simple HTML attributes, browser handles the rest. 😍

IMPORTANT ATTRIBUTES

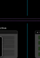
① popover
`<div popover>`
Tells browser:
"This element behaves like a popup."

② popovertarget
`<button popovertarget="menu">`
Tells browser:
"When button clicked, toggle popup having id="menu""

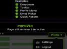
INTERNAL BROWSER FLOW 🔥

① Button Click
↓
② Browser searches target popover
↓
③ Browser moves popover to TOP LAYER
↓
④ Popup rendered above page
↓
⑤ Click outside
↓
⑥ Browser auto closes popup

TOP LAYER (Above everything)



CLICK OUTSIDE MAGIC 🔥



When user clicks outside popup:
✓ Browser auto closes it. WITHOUT JS.

REAL WORLD USE CASES

Profile Menus | Dropdowns | Emoji Pickers | Tooltips / Hints | Filters & Sort Menus | Notifications | And many more...

Modern Usage Note: <popover> makes building popups, tooltips, and floating UI simpler, faster, accessible, and more reliable — with zero heavy JavaScript.

<dialog> vs <popover>

★ Both are native browser popup systems, but their purpose is completely different.

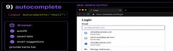
CORE IDEA

<dialog>
Used for: IMPORTANT interaction
Browser thinks:
"User must focus on this."
Examples:
- Login
- Payment
- Confirmation
- Alert
- Form Modal

<popover>
Used for: LIGHTWEIGHT floating UI
Browser thinks:
"This is just temporary helper UI."
Examples:
- Dropdown
- Tooltip
- Profile Menu
- Emoji Picker
- Quick Actions

VISUAL DIFFERENCE

DIALOG — Entire background becomes inactive | POPOVER — Page still remains interactive



User attention locked. | Small floating helper UI.

BIGGEST DIFFERENCE

Feature	<dialog>	<popover>
Purpose	Modal interaction	Floating UI
Background blocked	YES	NO
Focus trapped	YES	NO
User attention required	YES	NO
Usually centered	YES	Often near trigger
ESC close	YES	YES
Click outside close	Optional	Usually automatic
Used for forms	YES	Rarely
Lightweight	NO	YES

INTERNAL BROWSER BEHAVIOR

<dialog> Flow: showModal() → Browser creates modal state → Moves dialog to TOP LAYER → Background interaction disabled → Focus trapped inside dialog

<popover> Flow: Button clicked → Browser toggles popup → Moves popup to TOP LAYER → Page still interactive → Click outside → auto close

In short: <dialog> is for important, attention-demanding tasks. <popover> is for lightweight, helpful UI that doesn't interrupt the user.

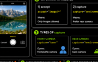
9) autocomplete

`<input autocomplete="email">`

Browser:
- autofill
- saved data
- smart suggestions

provide karta hai.

REAL WORLD OUTPUT



REAL WORLD
- login forms
- checkout forms
- signup systems

Login Form
Checkout Form
Signup Form
Login
Checkout
Create Account

10) capture ATTRIBUTE

`capture` is used with: `<input type="file">`

It tells mobile browser:
"Instead of opening gallery/files first, directly open camera or microphone."

SIMPLE EXAMPLE

```
<input
  type="file"
  capture>
```

REAL OUTPUT FLOW

WITHOUT capture: Click Upload ↓ Browser opens: • gallery • files • camera options

WITH capture: Click Upload ↓ Browser directly opens camera

① COMPLETE CAMERA EXAMPLE

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Capture Example</title>
</head>
<body>

  <h1>Open Camera</h1>

  <input
    type="file"
    accept="image/*"
    capture="environment">

</body>
</html>
```

WHAT HAPPENS?
On mobile:
Click input
↓
Rear camera opens
↓
Take photo
↓
Photo automatically selected

REAL WORLD OUTPUT (On Mobile)

WITHOUT app installation 🔥

② IMPORTANT ATTRIBUTES

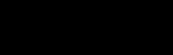
1) accept
`accept="image/*"`
Means:
Only images allowed

2) capture
`capture="environment"`
Means:
Prefer rear camera

③ TYPES OF capture

FRONT CAMERA
`capture="user"`
Opens:
front/selfie camera

REAR CAMERA
`capture="environment"`
Opens:
back camera



④ SELFIE EXAMPLE 🔥

```
<input
  type="file"
  accept="image/*"
  capture="user">
```

Used in:
- profile photo
- face verification
- selfie authentication

⑤ DOCUMENT SCANNER EXAMPLE 🔥

```
<input
  type="file"
  accept="image/*"
  capture="environment">
```

Used in:
- Aadhaar upload
- PAN card scan
- KYC verification
- QR scanners

⑥ AUDIO RECORDING EXAMPLE 🔥

```
<input
  type="file"
  accept="audio/*"
  capture>
```

Mobile may directly open:
Voice recorder / microphone

⑦ VIDEO RECORDING EXAMPLE 🔥

```
<input
  type="file"
  accept="video/*"
  capture>
```

Can directly open:
Video camera

⑧ REAL WORLD APPLICATIONS

Banking KYC | QR scanner apps | Document scanner | Face verification | Profile photo upload | Delivery proof systems | Expense receipt upload | Insurance claim apps

⑨ INTERNAL BROWSER FLOW

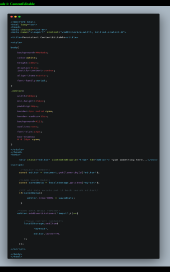
User taps input → Browser checks accept + capture → Browser talks to OS → OS camera app opens → Media captured → File returned to browser → Attached to input files

⑩ MOST IMPORTANT REALIZATION

Browser can communicate with:
Camera + Microphone + Gallery + Filesystem

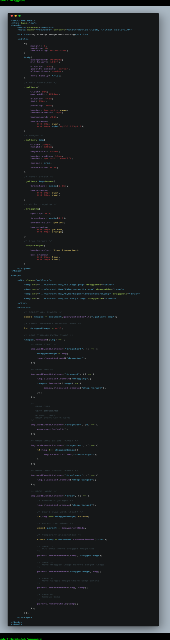
IMPORTANT LIMITATION
capture mainly works on mobile. Desktop browsers may ignore it, may just open file picker normally.

Code 1: ContentEditable



```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Persistent ContentEditable</title>

<style>

body{

    background:#0a0a0a;

    color:white;

    height:100vh;

    display:flex;
    justify-content:center;

    align-items:center;

    font-family:Arial;

}

.editor{

    width:500px;

    min-height:250px;

    padding:20px;

    border:3px solid cyan;

    border-radius:15px;

    background:#111;

    outline:none;

    font-size:22px;

    box-shadow:
    0 0 20px cyan;

}

</style>
</head>
<body>

     <div class="editor" contenteditable="true" id="editor"> Type something here...</div>

<script>

      /*SELECT ELEMENT*/
     const editor = document.getElementById("editor");


     /*LOAD SAVED DATA*/
     const savedData = localStorage.getItem("myText");


     /*If old data exists put it back inside editor*/
     if(savedData){

         editor.innerHTML = savedData;

      }


    /*SAVE DATA WHILE TYPING*/
    editor.addEventListener("input",()=>{


        /*Save current content*/
        localStorage.setItem(

              "myText",

              editor.innerHTML

        );

     });

</script>

</body>
</html>
```

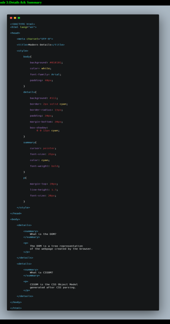
Code 2:Draggable



```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Drag & Drop Image Reordering</title>

    <style>

        *{
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        body{
            background: #0a0a0a;
            min-height: 100vh;

            display: flex;
            justify-content: center;
            align-items: center;

            font-family: Arial;
        }

        /* Main container */

        .gallery{

            width: 90%;
            max-width: 1200px;

            display: flex;
            gap: 25px;

            padding: 30px;

            border: 2px solid cyan;
            border-radius: 20px;

            background: #111;

            box-shadow:
                0 0 20px cyan,
                0 0 40px rgba(0,255,255,0.2);

        }

        /* Images */

        .gallery img{

            width: 220px;
            height: 220px;

            object-fit: cover;

            border-radius: 15px;
            border: 3px solid #00ffff;

            cursor: grab;

            transition: 0.3s;

        }

        /* Hover effect */

        .gallery img:hover{

            transform: scale(1.05);

            box-shadow:
                0 0 20px cyan,
                0 0 40px cyan;

        }

        /* While dragging */

        .dragging{

            opacity: 0.4;
```

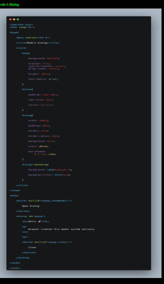
```
            });

            /* DROP LOGIC */

            img.addEventListener("drop", () => {

                /* Remove highlight */

                img.classList.remove("drop-target");

                /* Don't swap with itself */

                if(img === draggedImage) return;

                /* Parent container */

                const parent = img.parentNode;

                /* Temporary placeholder */

                const temp = document.createElement("div");

                /* STEP 1:
                   Put temp where dragged image was
                */

                parent.insertBefore(temp, draggedImage);

                /* STEP 2:
                   Move dragged image before target image
                */

                parent.insertBefore(draggedImage, img);

                /* STEP 3:
                   Move target image where temp exists
                */

                parent.insertBefore(img, temp);

                /* STEP 4:
                   Remove temp
                */

                parent.removeChild(temp);

            });

        });

    </script>

</body>
</html>
```

Code 3:Details && Summary



```
<!DOCTYPE html>
<html lang="en">

<head>

    <meta charset="UTF-8">

    <title>Modern Details</title>

    <style>

        body{

            background: #010101;

            color: white;

            font-family: Arial;

            padding: 40px;

        }

        details{

            background: #111;

            border: 2px solid cyan;

            border-radius: 15px;

            padding: 20px;

            margin-bottom: 20px;

            box-shadow:
                0 0 15px cyan;

        }

        summary{

            cursor: pointer;

            font-size: 25px;

            color: cyan;

            font-weight: bold;

        }

        p{

            margin-top: 20px;

            line-height: 1.7;

            font-size: 20px;

        }

    </style>

</head>

<body>

    <details>

        <summary>
            What is the DOM?
        </summary>

        <p>
            The DOM is a tree representation
            of the webpage created by the browser.
        </p>

    </details>

    <details>

        <summary>
            What is CSSOM?
        </summary>

        <p>
            CSSOM is the CSS Object Model
            generated after CSS parsing.
        </p>

    </details>

</body>

</html>
```

Code 4:Dialog



```
<!DOCTYPE html>
<html lang="en">

<head>

    <meta charset="UTF-8">

    <title>Modern Dialog</title>

    <style>

        body{

            background: #010101;

            display: flex;
            justify-content: center;
            align-items: center;

            height: 100vh;

            font-family: Arial;

        }

        button{

            padding: 15px 30px;

            font-size: 22px;

            cursor: pointer;

        }

        dialog{

            width: 400px;

            padding: 30px;

            border: none;

            border-radius: 20px;

            background: #111;

            color: white;

            box-shadow:
                0 0 25px cyan;

        }

        dialog::backdrop{

            background: rgba(0,0,0,0.7);

            backdrop-filter: blur(5px);

        }

    </style>

</head>

<body>

    <button onclick="popup.showModal()">

        Open Dialog

    </button>

    <dialog id="popup">

        <h1>Hello 🚀</h1>

        <p>
            Browser created this modal system natively.
        </p>

        <br>

        <button onclick="popup.close()">

            Close

        </button>

    </dialog>

</body>

</html>
```